1. **What is Python?**
   Python is a high-level, interpreted, general-purpose programming language. It supports multiple programming paradigms like procedural, object-oriented, and functional programming. It is known for its readability and concise syntax. Python is widely used in web development, data analysis, artificial intelligence, automation, and more.
2. **What are the key features of Python?**
   Python is easy to learn and use due to its simple syntax. It supports dynamic typing, automatic memory management, and a large standard library. Python is platform-independent and has extensive support for third-party packages. It's also open-source and has a vibrant community.
3. **What is PEP 8 and why is it important?**
   PEP 8 is Python's official style guide for writing clean and readable code. It recommends formatting rules like indentation, spacing, line length, and naming conventions. Following PEP 8 ensures consistency across Python codebases and improves collaboration among developers.
4. **What is the difference between list and tuple in Python?**
   A list is mutable, meaning its elements can be changed after creation, whereas a tuple is immutable. Lists use square brackets [ ], and tuples use parentheses ( ). Tuples are generally faster and used for fixed data. Lists are more flexible and used when frequent updates are needed.
5. **What are Python's data types?**
   Python supports several built-in data types like integers (int), floating-point numbers (float), strings (str), booleans (bool), lists, tuples, sets, and dictionaries. Each data type serves specific purposes and has associated methods for manipulation.
6. **What is a dictionary in Python?**
   A dictionary is an unordered, mutable collection of key-value pairs. Keys are unique and must be immutable, while values can be any data type. Dictionaries are defined using curly braces {} and allow fast access to values based on keys.
7. **How is memory managed in Python?**
   Python uses automatic memory management through reference counting and garbage collection. Objects are deleted when their reference count drops to zero. The garbage collector also handles cyclic references, ensuring efficient memory usage.
8. **What are Python's conditional statements?**
   Python supports if, elif, and else statements for decision-making. These statements evaluate expressions and execute blocks of code based on Boolean conditions. Indentation is critical in Python to define the code blocks correctly.
9. **What is the difference between is and == in Python?**
   The == operator compares the values of two objects, while is compares their identities (memory locations). If a == b is True, it means the data is the same; if a is b is True, it means both point to the same object in memory.
10. **What is a function in Python?**
    A function is a reusable block of code that performs a specific task. Functions are defined using the def keyword and can take parameters and return values. They help in organizing code and avoiding repetition.
11. **What are *args and **kwargs in Python?**
    *args allows a function to accept any number of positional arguments, while **kwargs lets it accept any number of keyword arguments. They are used when the number of inputs is not known in advance and help in flexible function calls.

12. **What is a lambda function?**
    A lambda function is an anonymous, one-line function defined using the lambda keyword. It can take any number of arguments but only one expression. Lambda functions are often used for short, throwaway operations like sorting or filtering.
13. **What are Python's loops?**
    Python supports for and while loops for iteration. The for loop is used to iterate over sequences like lists or strings, while while runs as long as a condition is true. Loops can be controlled using break, continue, and pass.
14. **What is the difference between break, continue, and pass?**
    break exits the loop completely, continue skips the current iteration and moves to the next, and pass does nothing and is used as a placeholder. They are used to control the flow inside loops and conditional statements.
15. **How do you handle exceptions in Python?**
    Exceptions are handled using the try, except, else, and finally blocks. Code that may raise an error is placed inside try, and the response is defined in except. finally runs regardless of exceptions and is used for cleanup.
16. **What is the difference between a shallow copy and a deep copy?**
    A shallow copy creates a new object but inserts references to the same elements, while a deep copy creates a new object with new copies of the elements. Shallow copies are created using copy() and deep copies using copy.deepcopy().
17. **What is list comprehension?**
    List comprehension provides a concise way to create lists using a single line of code. It includes an expression followed by a for loop, optionally with an if clause. It is more readable and faster than traditional for loops.
18. **What are Python modules and packages?**
    A module is a file containing Python code (functions, classes, variables), and a package is a directory containing multiple modules with an __init__.py file. They promote modular programming and code reuse.
19. **What is the __init__ method in Python?**
    The __init__ method is the constructor in Python classes. It is called automatically when a new object is created. It initializes instance variables and sets up the object's initial state.
20. **What is inheritance in Python?**
    Inheritance allows one class (child) to acquire properties and methods from another class (parent). Python supports single, multiple, multilevel, and hierarchical inheritance. It promotes code reuse and logical relationships between classes.
21. **What is polymorphism in Python?**
    Polymorphism allows the same function or method to behave differently depending on the context. In Python, this is commonly achieved through method overriding in classes and duck typing. For example, the same len() function works on strings, lists, and dictionaries. It improves flexibility and code reuse.
22. **What is encapsulation in Python?**
    Encapsulation is the practice of restricting access to certain parts of an object. It is implemented by defining private variables and methods using a single or double underscore prefix. This hides internal object details and provides controlled access via public methods, improving security and code integrity.

23. **What are Python decorators?**
    Decorators are functions that modify the behavior of other functions or methods without changing their code. They use the @decorator_name syntax and are often used for logging, access control, or timing functions. Decorators promote code reusability and separation of concerns.
24. **What are Python generators?**
    Generators are iterators that yield values one at a time using the yield keyword instead of returning all values at once. They are memory-efficient and ideal for working with large datasets or infinite sequences. Generators maintain state between iterations.
25. **What is the difference between a generator and an iterator?**
    An iterator is any object that implements the __iter__() and __next__() methods. A generator is a type of iterator created using a function with the yield keyword. Generators simplify iterator creation and retain state between yields automatically.
26. **What is a Python class and object?**
    A class is a blueprint for creating objects, which are instances of that class. Classes define attributes and methods, while objects hold specific data and behavior based on the class. Python uses the class keyword to define classes and __init__ for initialization.
27. **How does Python handle multiple inheritance?**
    Python supports multiple inheritance, where a class can inherit from more than one parent class. It uses the Method Resolution Order (MRO) to decide which method to invoke when names conflict. The MRO follows the C3 linearization algorithm to resolve ambiguities.
28. **What is the use of super() in Python?**
    The super() function is used to call methods from a parent or sibling class. It is commonly used in inheritance to invoke the __init__() method of the base class. This avoids explicitly naming the parent class and supports cooperative multiple inheritance.
29. **What are Python's built-in data structures?**
    Python includes several built-in data structures: lists, tuples, sets, and dictionaries. Each has unique properties: lists are ordered and mutable, tuples are ordered and immutable, sets are unordered and unique, and dictionaries store key-value pairs. They provide efficient data storage and manipulation.
30. **What is the difference between del, remove(), and pop()?**
    del deletes an item by index or deletes the entire object. remove() deletes the first occurrence of a specified value. pop() removes and returns an item at a given index (or the last item if no index is provided). Each serves different deletion use cases.
31. **What is slicing in Python?**
    Slicing is a way to extract a portion of a sequence like a list, string, or tuple using a start:stop:step format. For example, list[1:4] returns elements from index 1 to 3. Slicing provides an elegant and readable way to work with subsequences.
32. **How is exception handling done using try-except-finally?**
    In Python, risky code is placed in the try block, and potential errors are caught using except. The finally block runs regardless of exceptions and is used for cleanup actions. This structure ensures robust and fault-tolerant code execution.
33. **What is the difference between @staticmethod and @classmethod?**
    @staticmethod defines a method that doesn't access class or instance data. @classmethod takes cls as the first argument and can access class-level data. Static methods are used for

utility functions, while class methods are useful for alternative constructors or modifying class state.

34. **What is duck typing in Python?**
    Duck typing is a concept where the type of an object is less important than the methods it defines. If an object behaves like a certain type (e.g., has the right methods), it can be used as that type. "If it walks like a duck and quacks like a duck, it's a duck."
35. **What is the purpose of the with statement in Python?**
    The with statement simplifies resource management like file operations. It ensures that resources are automatically cleaned up, such as closing a file after reading. It is often used with context managers to reduce boilerplate and prevent resource leaks.
36. **What is a Python context manager?**
    A context manager is an object that defines __enter__() and __exit__() methods. It sets up a context (like opening a file) and tears it down automatically (like closing the file) after execution. It is used with the with statement for cleaner resource handling.
37. **How does Python's isinstance() function work?**
    The isinstance() function checks if an object belongs to a particular class or a tuple of classes. It returns True if the object is an instance of the given class. It is preferred over type-checking using type() in inheritance scenarios.
38. **What are magic methods in Python?**
    Magic methods (also called dunder methods) are special methods that start and end with double underscores, like __init__, __str__, and __len__. They enable operator overloading and define object behavior. For example, __add__ allows custom behavior for the + operator.
39. **What is list unpacking in Python?**
    List unpacking allows assigning values from a list or tuple to multiple variables in a single line. For example, a, b = [1, 2] assigns 1 to a and 2 to b. It improves code readability and is often used in functions returning multiple values.
40. **How do you reverse a list in Python?**
    You can reverse a list using list.reverse(), which modifies the list in place, or by slicing like list[::-1], which returns a reversed copy. The reversed() function can also be used to get an iterator. Each approach is useful based on the desired behavior.
41. **How do you copy an object in Python?**
    To copy an object in Python, you can use the copy module. A shallow copy is created using copy.copy(), which copies the object but not nested objects. A deep copy, created with copy.deepcopy(), copies everything including nested structures. It's crucial to choose the right type of copy depending on whether your object has references to other objects inside it.
42. **What is the difference between mutable and immutable types?**
    Mutable types, like lists, sets, and dictionaries, can be changed after their creation—items can be added, modified, or deleted. Immutable types, such as strings, tuples, and integers, cannot be modified once created. Any operation on immutable objects results in a new object. Understanding mutability helps avoid unintended side effects.
43. **What is the use of the id() function in Python?**
    The id() function returns the memory address (identity) of an object. It helps to compare the identity of two objects using the is operator. If two variables have the same id, they refer to the same object in memory. This is useful for debugging and understanding object behavior.
44. **What are Python's built-in functions?**
    Python provides many built-in functions like len(), max(), min(), sum(), type(), input(), print(),

range(), and many more. These functions perform common tasks and eliminate the need to write repetitive code. They improve development speed and reduce bugs by using well-tested logic.

45. **What is the difference between range() and xrange()?**
    In Python 2, range() returns a list, while xrange() returns a generator-like object that produces numbers on demand. In Python 3, xrange() is removed and range() behaves like the old xrange(). It's memory-efficient and ideal for loops over large sequences without consuming extra memory.
46. **How are arguments passed in Python: by reference or by value?**
    Python uses a model known as "call by object reference" or "call by sharing." If you pass a mutable object, changes inside the function affect the original. If the object is immutable, changes won't affect the original. Understanding this helps avoid unexpected behavior in functions.
47. **What is a Python module?**
    A module is a file containing Python definitions and statements. It can include functions, variables, and classes, which can be imported and reused in other programs using the import statement. Python's standard library is full of useful modules like math, datetime, and os.
48. **What is a Python package?**
    A package is a directory containing multiple Python modules and an __init__.py file. It helps in organizing code into a hierarchical structure. Packages promote modular programming, making large codebases easier to manage and reuse. You import them using the dot notation like package.module.
49. **How do you install third-party packages in Python?**
    Python uses pip, the package installer, to download and install third-party packages from the Python Package Index (PyPI). You can install a package using pip install package_name. Virtual environments are often used to manage dependencies per project and avoid conflicts between them.
50. **What is a virtual environment in Python?**
    A virtual environment is an isolated Python environment for a specific project. It allows you to install packages without affecting the global Python installation. Tools like venv and virtualenv are used to create these environments. This ensures consistent dependencies across development, testing, and production.
51. **What is recursion in Python?**
    Recursion is when a function calls itself to solve smaller instances of a problem. Each call pushes a new frame to the call stack, and recursion continues until a base case is met. It's elegant for tasks like tree traversal or factorial calculation, but can lead to stack overflow if not controlled.
52. **What is the use of assert in Python?**
    The assert statement tests if a condition is true. If it isn't, an AssertionError is raised, optionally with a custom message. It's commonly used for debugging and validating code during development. However, assertions should not be used for handling run-time exceptions in production code.
53. **What are Python's loop control statements?**
    Python includes break, continue, and pass for controlling loop execution. break exits the loop entirely, continue skips the current iteration, and pass does nothing (used as a placeholder). These tools help refine loop logic and create clean, efficient control flows.

54. **What is the difference between global and nonlocal keywords?**
The global keyword is used to declare that a variable inside a function refers to a global variable. The nonlocal keyword is used to work with variables in the nearest enclosing (non-global) scope. These keywords help in modifying outer-scope variables within nested functions.
55. **What is the Python Global Interpreter Lock (GIL)?**
The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. This simplifies memory management but limits parallelism in multi-threaded programs. To achieve true concurrency, Python developers often use multiprocessing instead of multithreading.
56. **How do you open and read a file in Python?**
You can use the open() function to open a file, and read(), readline(), or readlines() to read its contents. Always use the with statement to handle files, which ensures automatic closing. File modes like 'r', 'w', 'a', and 'b' determine the access behavior.
57. **What is the difference between text and binary files in Python?**
Text files contain readable characters and use encodings like UTF-8, while binary files store raw bytes. You open text files using modes like 'r', and binary files with 'rb'. Proper handling ensures that files are read and written without corruption or data loss.
58. **How do you write data to a file in Python?**
Use the open() function with write mode ('w' or 'a') and the write() or writelines() method. It’s best to use the with block to avoid file leaks. Writing overwrites existing content unless you open the file in append mode, which adds new content to the end.
59. **What are Python string methods?**
Python provides many built-in string methods such as lower(), upper(), strip(), replace(), find(), and split(). These methods do not modify the original string but return new ones. They are used for string manipulation and cleaning, especially in text-processing applications.
60. **How do you format strings in Python?**
Python supports string formatting using f-strings (f"Name: {name}"), the format() method, and the % operator. F-strings are the most modern and preferred due to readability and performance. Formatting helps create dynamic, readable, and well-structured output for users.
61. **What are Python lambda functions?**
Lambda functions are anonymous, single-expression functions defined using the lambda keyword. They can have any number of arguments but only one expression. Commonly used for short, throwaway functions, especially with map(), filter(), or sorted(). For example: lambda x: x + 5. Though concise, they should be used sparingly to keep code readable.
62. **How do map(), filter(), and reduce() work in Python?**
map() applies a function to all items in an iterable and returns a map object. filter() returns items for which the function returns True. reduce() (from functools) cumulatively applies a function to the iterable. These functions support a functional programming style and often reduce boilerplate code when used appropriately.
63. **What is list comprehension in Python?**
List comprehension offers a concise way to create lists using a single line of code. It replaces the need for loops to build lists. Syntax: [expression for item in iterable if condition]. It's readable and often more efficient. Example: [x*x for x in range(5) if x % 2 == 0].

64. What is dictionary comprehension in Python?
Dictionary comprehension allows creating dictionaries in a single line using a similar syntax as list

comprehension. Syntax: {key_expr: value_expr for item in iterable if condition}. It enhances readability and can replace longer loops for building dictionaries. Useful in data transformation tasks.

65. What is the purpose of enumerate() in Python?
The enumerate() function adds a counter to an iterable and returns it as an enumerate object. It's commonly used in loops when you need both index and value. For example: for i, val in enumerate(list):. It eliminates the need for range(len(list)), making code more Pythonic.

66. How does zip() work in Python?
The zip() function combines multiple iterables element-wise into tuples. It returns a zip object which can be converted into a list or iterator. It stops at the shortest iterable. Useful for iterating over multiple sequences together, such as combining names and scores.

67. What are Python's bitwise operators?
Bitwise operators work at the binary level. These include & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), and >> (right shift). They are often used in low-level programming, cryptography, and optimizing operations involving flags or masks. Example: 5 & 3 yields 1.

68. What is the difference between is and == in Python?
The == operator checks value equality, meaning whether two variables have the same data. The is operator checks identity, i.e., whether both variables point to the same object in memory. While == can return True even if the objects are different, is is stricter.

69. What is monkey patching in Python?
Monkey patching refers to modifying or extending code at runtime without changing the original source. It allows replacing methods or attributes of classes or modules dynamically. While powerful, it can lead to maintenance issues and unpredictable behavior, so it should be used cautiously.

70. What are Python memory management features?
Python handles memory management automatically using reference counting and garbage collection. Objects are stored in private heap space, and the gc module can be used to interact with the garbage collector. Memory for small integers and strings is also optimized through interning.

71. What is slicing with negative indices in Python?
Negative indices allow slicing from the end of a sequence. For example, list[-1] returns the last element, and list[-3:-1] slices the third-last to second-last. It's a powerful feature for accessing elements in reverse or manipulating sequences without needing to reverse them first.

72. What is the purpose of *args and **kwargs?
*args allows a function to accept any number of positional arguments, while **kwargs allows it to accept any number of keyword arguments. They are useful for writing flexible functions and passing variable-length arguments. They can also be unpacked when calling other functions.

73. What is the __init__ method in Python classes?
The __init__ method is the constructor in Python and is called automatically when a class object is created. It initializes the object's attributes and allows passing custom values during instantiation. Though not mandatory, it's crucial for defining initial object behavior.

74. What is the difference between __str__ and __repr__?
__str__() returns a user-friendly string representation of an object, while __repr__() returns an unambiguous string for developers. If __str__ is not defined, Python falls back to __repr__. Typically, __repr__ should return a string that can recreate the object.

75. What is operator overloading in Python?
Operator overloading lets you redefine how operators behave for custom classes by implementing special methods like __add__, __sub__, or __eq__. This allows intuitive use of operators on user-defined types. It improves readability but should be used responsibly to avoid confusion.

76. What are Python slots (__slots__)?
__slots__ is a mechanism to limit the attributes of class instances and reduce memory usage by preventing the creation of __dict__. It can improve performance when creating many objects. However, it restricts dynamic addition of new attributes to instances.

77. What is the purpose of yield in Python?
The yield keyword is used in generators to produce a value and pause the function, resuming from that point on the next call. It is useful for iterating over large datasets without loading everything into memory. It also helps build stateful iterators in a clean way.

78. What are Python descriptors?
Descriptors are classes that define methods like __get__, __set__, and __delete__ and are used to customize attribute access. They are the foundation of Python's property system and help manage computed properties or enforce validation. Used in advanced object-oriented designs.

79. What are Python metaclasses?
Metaclasses are classes of classes that define how classes behave. Using a metaclass, you can control class creation by modifying attributes or inheritance. They are defined using the metaclass keyword in a class and are often used for frameworks, ORM models, and API scaffolding.

80. How do you handle JSON data in Python?
Python's json module is used to parse JSON strings into Python dictionaries using json.loads() and convert Python objects into JSON strings using json.dumps(). It is widely used in APIs, configuration files, and data interchange. The module ensures easy integration with web services.

81. What is duck typing in Python and how does it apply to functions?
Duck typing means that Python cares more about what an object can do rather than what it is. If a function expects an object with certain methods or behavior, any object that provides them will work. This makes functions more generic and promotes loose coupling.

82. What is the purpose of the any() and all() functions?
any() returns True if at least one element in an iterable is True. all() returns True only if all elements are True. These functions are handy for validating conditions across a collection. They improve code clarity and reduce the need for verbose loops.

83. How do you merge two dictionaries in Python?
You can merge dictionaries using the unpacking operator {**dict1, **dict2} or with the update() method. In Python 3.9+, the | operator is used: dict3 = dict1 | dict2. If keys overlap, the second dictionary's values will overwrite the first.

84. What is a frozenset in Python?
A frozenset is an immutable version of a set. It supports all standard set operations like union, intersection, and difference but cannot be modified (no add or remove). Useful for creating hashable set objects that can be used as dictionary keys.

85. What is the use of the collections module in Python?
The collections module provides specialized container datatypes like deque, Counter, OrderedDict, defaultdict, and namedtuple. These offer more functionality or better performance for specific use cases than built-in types. It's a standard and efficient way to manage structured data.

86. How do you handle time and date in Python?
Python has the datetime module to handle dates and times. It supports date arithmetic, formatting, parsing, and timezone handling. You can create datetime objects using datetime.datetime.now() or manipulate with methods like .strftime() and .timedelta().

87. What is a namedtuple in Python?
A namedtuple is a subclass of tuple from the collections module. It allows you to access fields by name instead of index. This makes code more readable and self-documenting. It’s a lightweight alternative to classes when storing small, immutable records.

88. What is the purpose of Python’s re module?
The re module supports regular expressions for pattern matching in strings. You can use functions like match(), search(), findall(), and sub() to work with text data. It’s powerful for data validation, extraction, and transformation in text processing tasks.

89. What is the difference between shallow and deep copy?
A shallow copy duplicates only the outer object, not nested objects. Changes to nested elements affect both copies. A deep copy creates a new object and recursively copies all nested objects. Use copy.copy() for shallow and copy.deepcopy() for deep copies from the copy module.

90. How is exception propagation handled in Python?
If an exception is not caught in the function where it occurs, it propagates up the call stack until it is caught or reaches the top level, terminating the program. You can use multiple except blocks to handle specific exceptions and finally for cleanup actions.

91. What is the difference between classmethod and staticmethod?
A classmethod receives the class as the first argument and can modify class-level data. It’s defined using @classmethod. A staticmethod does not receive any implicit first argument and behaves like a normal function inside a class. It is defined with @staticmethod. Use class methods for factory patterns and static methods for utility operations.

92. What is the use of the property() function in Python?
The property() function is used to create managed attributes in classes. It allows a method to be accessed like an attribute while enabling getter, setter, and deleter functionality. This helps encapsulate logic for accessing private variables. It's often used with decorators like @property, @<property>.setter.

93. How do you create custom exceptions in Python?
Custom exceptions are created by subclassing the built-in Exception class. You can define your own

__init__ or __str__ methods if needed. This helps provide specific error types for better debugging and control flow. Example: class MyError(Exception): pass.

94. What is Python’s with statement used for?

The with statement simplifies resource management by automatically handling setup and teardown. It is mostly used with file operations or database connections. The context manager ensures resources like files are properly closed, even if an exception occurs. It's more concise and safer than manual open/close.

95. What are iterators in Python?

An iterator is an object that implements the __iter__() and __next__() methods. It allows sequential access to elements without exposing the underlying structure. Iterators are memory-efficient and used with loops like for. Once exhausted, they cannot be reset unless reinitialized.

96. What are generators in Python?

Generators are special iterators created using functions with the yield keyword. Each call to next() resumes execution where it last left off. Generators are memory-efficient and great for streaming large data or creating infinite sequences. Unlike lists, they don’t store all items in memory.

97. What is the purpose of the assert statement in Python?

The assert statement tests if a condition is True, and raises an AssertionError if not. It's used mainly for debugging and internal consistency checks during development. Assertions can be globally disabled with the -O optimization flag. They should not be used for handling runtime errors.

98. What are Python coroutines and how are they different from generators?

Coroutines are similar to generators but can consume data using yield expressions and use await in async code. While generators produce data, coroutines are used for cooperative multitasking. Coroutines use async def and are typically used in asynchronous programming for non-blocking operations.

99. How is multithreading different from multiprocessing in Python?

Multithreading runs multiple threads within a process and is suitable for I/O-bound tasks. However, due to the Global Interpreter Lock (GIL), threads cannot run Python bytecode in true parallelism. Multiprocessing spawns separate processes and achieves parallel execution, ideal for CPU-bound tasks.

100. What is the Global Interpreter Lock (GIL) in Python?

The GIL is a mutex that allows only one thread to execute Python bytecode at a time. It simplifies memory management but limits parallel execution in multi-threaded programs. For true parallelism in Python, multiprocessing is recommended over threading, especially for CPU-bound work.

101. What are Python’s built-in data types?

Python has several built-in data types including numeric types (int, float, complex), sequence types (list, tuple, range), mapping (dict), set types (set, frozenset), boolean (bool), and text (str). These types support a variety of operations and methods for data manipulation.

102. What are Python’s sequence types?

Sequence types in Python include list, tuple, range, and str. They allow element access via indexing,

slicing, and support iteration. Lists and tuples are commonly used for storing ordered data, while range is typically used in loops. Strings are immutable sequences of characters.

103. How do you remove duplicates from a list in Python?
You can remove duplicates using set(), which automatically removes repeated elements: list(set(mylist)). However, this approach does not preserve order. To maintain order, use dict.fromkeys(mylist) or a loop with a helper set. Libraries like pandas also offer drop_duplicates().

104. What is the difference between mutable and immutable types in Python?
Mutable types (like list, dict, set) can be changed after creation, while immutable types (like int, str, tuple) cannot. Modifying an immutable type results in a new object. Understanding this helps avoid side-effects, especially when passing objects to functions.

105. What are Python's numeric types and their characteristics?
Python supports int (arbitrary precision), float (double-precision), and complex (real and imaginary parts). Integers have no size limit, floats support decimals and scientific notation, and complex numbers use the j suffix. These types are interoperable with arithmetic operations.

106. How do you reverse a list in Python?
You can reverse a list using list.reverse() (in-place), slicing (list[::-1]), or reversed(list) (returns an iterator). Choose the method based on whether you want to modify the original list or return a new one. Reversal is often used in sorting and traversal algorithms.

107. How do you check the memory size of an object in Python?
You can use sys.getsizeof() from the sys module to check the memory size of an object. This function returns the size in bytes. Keep in mind it does not account for referenced objects recursively. For deeper analysis, use third-party libraries like pympler.

108. What is the difference between range() and xrange()?
In Python 2, range() creates a list, while xrange() returns an iterator and is more memory-efficient. In Python 3, range() behaves like xrange() from Python 2, generating numbers lazily. Python 3 no longer includes xrange() as it's been fully replaced.

109. What is a memoryview object in Python?
A memoryview object allows access to the internal data of an object that supports the buffer protocol without copying it. It's efficient for large binary data manipulation like images or streams. You can slice and modify data directly using this object, improving performance.

110. What is the use of the id() function in Python?
The id() function returns the unique identity (memory address) of an object during its lifetime. It is used to check object identity and compare references. In CPython, it corresponds to the object's memory location. It helps in debugging and understanding object sharing.

111. What are Python's magic methods?
Magic methods (also known as dunder methods) start and end with double underscores, like __init__, __str__, __len__. They enable operator overloading and special behavior for objects. For example, __add__ lets you define custom addition behavior. These are key to Python's object model.

112. What is the __name__ == "__main__" idiom?
This idiom ensures that code is only executed when the file is run directly, not when imported. Python assigns __name__ = "__main__" to the top-level script. It is commonly used to wrap test code or main functions, providing modularity and reusability across files.

113. How do you handle circular imports in Python?
Circular imports occur when two modules import each other directly or indirectly. To resolve them, restructure the code, use import statements inside functions, or refactor into a common module. Circular imports can cause runtime errors or attribute access issues.

114. What are abstract base classes in Python?
Abstract Base Classes (ABCs) define interfaces that must be implemented by derived classes. Python provides the abc module to create ABCs using @abstractmethod. They enforce a structure in OOP design and prevent instantiation of incomplete subclasses. Useful for APIs and frameworks.

115. How do you prevent a class from being subclassed in Python?
To prevent subclassing, override the __init_subclass__ method and raise an exception inside it. Alternatively, use metaclasses to block subclassing. While Python doesn't have a built-in final keyword like Java, such mechanisms can enforce similar behavior.

116. What is the use of globals() and locals() in Python?
globals() returns a dictionary of the current global symbol table, while locals() returns a dictionary of the local namespace. They are useful for dynamic code execution, debugging, and reflection. However, their output may differ depending on the context of the call.

117. What is the purpose of eval() and exec() functions?
eval() evaluates a Python expression and returns the result, while exec() executes Python code dynamically. They are powerful but risky, especially with untrusted input. For example, eval("2 + 2") returns 4, whereas exec("x = 5") defines a variable in the namespace.

118. What is the difference between deepcopy() and copy()?
copy() creates a shallow copy of an object, duplicating only the outer object. deepcopy() recursively copies all nested objects to create a fully independent clone. Use deepcopy() from the copy module when working with nested structures that require isolation.

119. How does Python handle type conversion?
Python performs both implicit and explicit type conversions. Implicit conversion is done automatically (e.g., int + float becomes float). Explicit conversion uses functions like int(), str(), or float(). It's important for avoiding type errors in mixed-type operations.

120. How do you compare two dictionaries in Python?
You can compare dictionaries using the == operator, which checks both keys and values. Order doesn't matter in dictionary comparison. For deeper comparison, use loops or libraries like deepdiff. Dictionaries must be of the same length and content to return True.

121. What is duck typing in Python?
Duck typing is a concept in Python where the type of an object is less important than the methods or behaviors it supports. The term comes from the saying: “If it looks like a duck and quacks like a duck, it's a duck.” Python encourages this style by not enforcing strict type checking. Instead, you just call

methods or access attributes assuming they exist. If the object supports the operation, it works; otherwise, an error occurs. This makes code more flexible and reusable. For example, a function that accepts any object with a len() method doesn't care if it's a string, list, or custom class. Duck typing is a form of polymorphism. It helps write more generic and extensible code.

122. What are descriptors in Python?

Descriptors are objects that define how attribute access is interpreted by Python. They are classes that implement any of the descriptor methods: __get__, __set__, or __delete__. Descriptors allow custom behavior when accessing or setting attributes. They are used internally by Python for functions, methods, and properties. You can use descriptors to implement features like type checking, logging, or computed properties. They provide a powerful way to control access to class attributes. The most common example of a descriptor is the property() function. Descriptors can be reused across classes, promoting cleaner and more modular code.

123. What is the purpose of metaclasses in Python?

Metaclasses are classes of classes. They define how classes behave. Just as classes define how objects behave, metaclasses define how classes themselves are created. You can use a metaclass to modify or customize class creation dynamically. By default, type is the built-in metaclass in Python. Metaclasses are useful when you want to enforce coding standards, register classes automatically, or inject methods and attributes. You define a metaclass by inheriting from type and overriding __new__ or __init__. To apply a metaclass, use the metaclass= keyword in class definition. They are an advanced tool for framework development.

124. What are Python closures?

Closures are functions that remember the environment in which they were created. When a nested function refers to a variable from its enclosing scope, and that function is returned, it becomes a closure. Closures allow encapsulation of behavior and state together. They are commonly used in decorators, callbacks, and factories. Since the enclosed variables persist beyond the lifetime of the outer function, closures can help maintain state in a functional style. Closures are a part of Python's support for first-class functions. You can inspect closures using the __closure__ attribute. They are powerful but should be used carefully to avoid unexpected behavior.

125. How do you implement data hiding in Python?

Python doesn't enforce strict access modifiers like private or protected. However, data hiding can be done using naming conventions. Prefixing an attribute with a single underscore (e.g., _var) indicates it's intended for internal use. Prefixing with double underscores (e.g., __var) triggers name mangling, making it harder to access from outside the class. While this doesn't make it truly private, it reduces accidental access. Properties and getter/setter methods can further control access. Python trusts the developer to respect these conventions. Encapsulation can still be achieved effectively with these mechanisms.

126. What is monkey patching in Python?

Monkey patching is the practice of dynamically modifying classes or modules at runtime. It allows you to change behavior without modifying the original source code. This is useful in testing, debugging, or extending third-party libraries. For example, you can replace a method of a class with a new one at runtime. However, monkey patching can lead to maintenance issues and unexpected behavior if overused. It breaks encapsulation and may make the codebase harder to understand. It's considered a powerful but risky technique. Always document monkey patches thoroughly when used.

127. What is the difference between shallow copy and deep copy in Python?

A shallow copy creates a new object but inserts references to the original objects inside it. Changes to nested objects affect both the original and the copy. A deep copy recursively copies all objects, creating completely independent objects. Python provides the copy module with copy() and deepcopy() functions. Use copy() for simple, flat structures and deepcopy() for complex, nested ones. Deep copies consume more memory and are slower but prevent shared references. Understanding this difference helps avoid unintended side-effects when copying collections.

128. What is the difference between __str__ and __repr__ in Python?

__str__ is used to define the human-readable string representation of an object, used by str() and print(). __repr__ is for the official representation, used by repr() and meant for developers. If __str__ is not defined, Python falls back to __repr__. The goal of __repr__ is to return a string that can ideally recreate the object. You can override both in custom classes to improve debugging and output clarity. Example: __repr__ = "Person('John', 30)", __str__ = "John, age 30". Use them to control how your objects appear.

129. How does Python's garbage collection work?

Python uses automatic memory management via reference counting and cyclic garbage collection. Every object keeps a reference count; when it reaches zero, the object is deallocated. For cyclic references (like two objects referring to each other), Python uses a garbage collector in the gc module. The collector detects unreachable cycles and frees them periodically. Developers can trigger or disable garbage collection manually. While mostly invisible, understanding it helps prevent memory leaks in long-running applications. You can inspect or debug memory issues using tools like gc, tracemalloc, or external profilers.

130. What are context managers in Python?

Context managers manage resources such as files, sockets, or locks using the with statement. They ensure proper acquisition and release of resources, even if exceptions occur. A context manager implements __enter__ and __exit__ methods. Python’s with open() is a common example. Custom context managers can be created using classes or the contextlib module. They help avoid resource leaks and make code cleaner. Using with guarantees resources are closed or cleaned up in a predictable way, making context managers an essential tool for robust applications.

131. What is the difference between @staticmethod and @classmethod in Python?

@staticmethod defines a method that does not receive an implicit first argument (like self or cls). It behaves like a regular function but lives inside the class namespace. Use it when the method does not modify the class or instance state.

@classmethod, on the other hand, receives the class (cls) as the first argument and can modify class-level attributes. It is often used for alternative constructors or factory methods. Both decorators make methods callable on the class itself. Use @staticmethod when behavior is independent of class, and @classmethod when you need to access or modify class state.

132. How do you handle memory management in Python?

Memory management in Python is handled by the Python memory manager, which includes private heap space and garbage collection. Objects and data structures are stored in this private heap. Reference counting is the primary technique for deallocating unused memory.

When reference counts drop to zero, the object is deleted. For circular references, Python uses a cyclic garbage collector via the gc module. Developers can use gc.collect() to trigger collection

manually and tools like tracemalloc to track memory usage. Proper coding practices like closing files and breaking reference cycles help optimize memory.

133. What is the difference between a module and a package in Python?

A **module** is a single Python file (.py) containing functions, classes, or variables. It allows for code reuse and logical separation. You can import a module using the import statement.

A **package** is a directory that contains multiple modules and a special __init__.py file, which tells Python it's a package. Packages support a hierarchical organization of modules, allowing for large projects to be broken into manageable pieces. Use packages when structuring large applications. Modules are the building blocks of packages.

134. How does Python handle multiple inheritance?

Python supports multiple inheritance, allowing a class to inherit from more than one parent class. This means a class can combine functionalities from multiple classes. The method resolution order (MRO) decides the order in which base classes are searched when calling a method.

Python uses the C3 linearization algorithm to determine this order, which can be viewed using ClassName.__mro__. Diamond problems are resolved cleanly using MRO. While powerful, multiple inheritance should be used carefully to avoid complexity. When in doubt, consider composition over inheritance.

135. What are properties in Python and how are they used?

Properties in Python allow you to customize access to instance attributes using getter, setter, and deleter methods. They are defined using the built-in property() function or the @property decorator.

The main benefit is encapsulation — you can make an attribute read-only, compute its value dynamically, or validate data before setting it. This allows for more flexible and controlled access without changing the external API. Properties make attribute access look like direct variable usage while still providing method-level control.

136. What is the with statement in Python and how does it work?

The with statement simplifies resource management. It is commonly used with file handling, database connections, and threading locks. When used, the __enter__ method of a context manager is called before the block starts, and __exit__ is called afterward, even if exceptions occur.

This ensures that resources are released properly, avoiding memory leaks and other issues. For example, with open("file.txt") as f: ensures the file is closed automatically. The contextlib module also provides utilities for creating context managers easily.

137. What is the purpose of __slots__ in Python classes?

The __slots__ declaration in a class restricts the creation of arbitrary new attributes and reduces memory usage. Normally, each object has a __dict__ to store attributes, but using __slots__ prevents this.

Instead, space is allocated only for the declared attributes. This speeds up attribute access and lowers memory overhead, especially in large numbers of small objects. However, using __slots__ removes some flexibility, such as adding new attributes dynamically. It is ideal for performance optimization in constrained environments.

138. What is the Global Interpreter Lock (GIL) in Python?

The Global Interpreter Lock (GIL) is a mutex that protects access to Python objects, ensuring that only one thread executes Python bytecode at a time. It simplifies memory management in CPython

but limits the effectiveness of multi-threading.
As a result, CPU-bound tasks do not gain much from threading in Python. To overcome this, developers use multiprocessing (which spawns separate processes) or alternative interpreters like Jython or IronPython. The GIL is less of an issue for I/O-bound applications like web servers or file I/O.

139. How can you improve the performance of a Python application?
To optimize Python performance, first profile the code using tools like cProfile or line_profiler to identify bottlenecks. Use built-in functions and libraries, which are faster and written in C.
Avoid unnecessary computations, minimize global variable access, and use generators for large data sets. Data structures like sets and dictionaries can improve search speed. Consider using NumPy for numerical computations and multiprocessing for parallel execution. In critical areas, write extensions in Cython or compile using PyPy.

140. What are coroutines in Python and how do they differ from generators?
Coroutines are generalizations of generators. While both use yield, coroutines use yield for both sending and receiving data, allowing more complex control flow. Coroutines are defined with async def and use await for non-blocking calls.
They are used for cooperative multitasking and are fundamental in asynchronous programming in Python. Unlike generators, coroutines can pause at await expressions and resume later, making them ideal for I/O-bound or high-concurrency programs. The asyncio module provides a complete framework for coroutine-based programming.

141. What is the difference between iterators and generators in Python?
An **iterator** is any object that implements the __iter__() and __next__() methods, allowing iteration one element at a time. You can create custom iterators using classes.
A **generator**, on the other hand, is a special type of iterator created using a function with the yield keyword. Each time yield is called, the function pauses its state until the next value is requested.
Generators are more memory-efficient as they yield values lazily (on-demand). While all generators are iterators, not all iterators are generators. Generators are useful for large data sets or infinite sequences and are easier to write than class-based iterators.

142. How does exception handling work in Python?
Python uses try-except blocks to catch and handle exceptions. Code that might raise an error is placed inside the try block. If an exception occurs, Python skips the rest of the block and jumps to the except clause.
You can catch specific exceptions like ZeroDivisionError, or use a generic except clause. else can be used to run code if no exception occurs, and finally runs code regardless of whether an exception was raised.
Raising exceptions manually is done using raise. Proper exception handling ensures graceful program termination and error logging.

143. What is memoization and how is it implemented in Python?
Memoization is an optimization technique where function results are cached so repeated calls with the same arguments return instantly. This avoids redundant computations, especially in recursive functions.
Python provides the functools.lru_cache() decorator to implement memoization easily. It caches the results of a function up to a specified number of recent calls.

For example, recursive Fibonacci functions benefit significantly from memoization. It improves performance but uses additional memory, so it’s ideal for functions with overlapping subproblems.

144. What are Python's magic methods?

Magic methods (also called dunder methods) are special methods in Python that start and end with double underscores, like __init__, __str__, __add__, etc. They enable operator overloading and object customization.

For instance, __len__ allows an object to respond to len(), while __getitem__ lets it support indexing. Magic methods control class behavior in expressions, comparisons, conversions, and context management.

By overriding them, you make your objects behave like built-in types. They enhance readability and consistency when designing custom classes.

145. How is exception chaining handled in Python?

Python supports **exception chaining** to preserve the original traceback when one exception causes another. This is done using the raise ... from ... syntax.

For example: raise ValueError("Invalid input") from original_exception. This helps debug root causes when handling exceptions at multiple levels.

The chained exceptions are displayed using __cause__ or __context__ attributes. It is especially helpful in layered code, like libraries or APIs, where you catch and re-raise exceptions with more context.

146. What is the difference between a shallow copy and deep copy of a list?

A **shallow copy** of a list creates a new list object, but its elements are still references to the same inner objects. Changes to nested objects affect both lists.

In contrast, a **deep copy** recursively copies all nested elements, resulting in an entirely independent clone. Python's copy module provides both: copy.copy() for shallow copy and copy.deepcopy() for deep copy.

Use shallow copy for flat structures and deep copy when your data contains nested mutable objects. Understanding the difference is key to avoiding shared-state bugs.

147. What is the role of the __init__.py file in Python packages?

The __init__.py file indicates to Python that the directory should be treated as a package. In Python 2.x, it was mandatory, while in Python 3.3+, it’s optional but still commonly used.

It can be empty or contain initialization code, import statements, or package metadata. It’s often used to define the __all__ list, controlling what’s exposed via from package import *.

Using __init__.py helps manage namespace, control imports, and structure modular applications effectively.

148. What is the difference between is and == in Python?

The == operator compares the **values** of two objects and returns True if they are equal in content. The is operator checks whether two references point to the **same object** in memory.

For example, two strings with the same content may be equal (==) but not identical (is) unless they’re interned. is is used for checking object identity, like if x is None:.

Understanding the difference is crucial in avoiding bugs when comparing mutable objects like lists or dictionaries.

149. What is the purpose of functools.partial() in Python?

functools.partial() is used to fix a certain number of arguments of a function and generate a new function with fewer parameters. It’s useful for creating specialized versions of functions without rewriting them.

For example, partial(pow, 2) returns a function that squares numbers. It helps improve code modularity, readability, and reusability.

Partial functions are often used in callbacks, GUI programming, and functional programming where functions are passed around with pre-filled parameters.

150. What are Python's built-in data types?

Python includes several built-in data types grouped into categories. Numeric types include int, float, and complex. Sequence types are list, tuple, and range.

Text is represented by str. Set types include set and frozenset. Dictionary is dict. Boolean is bool. Binary types are bytes, bytearray, and memoryview.

These types form the core of Python data manipulation. Each has associated methods for transformation, searching, and sorting. Understanding their behaviors helps in effective programming.